

Nama : Achmad Pradita Dwi Firmansyah
NIM : 254107020130
Kelas / Absen : TI-1G / 01

JOBSHEET 14

Tree

Tugas Praktikum

1. Buat method di dalam class **BinaryTree00** yang akan menambahkan node dengan cara rekursif (**addRekursif()**).
2. Buat method di dalam class **BinaryTree00** untuk menampilkan data mahasiswa dengan IPK paling kecil dan IPK yang paling besar (**cariMinIPK()** dan **cariMaxIPK()**) yang ada di dalam binary search tree.
3. Buat method dalam class **BinaryTree00** untuk menampilkan data mahasiswa dengan IPK di atas suatu batas tertentu, misal di atas 3.50 (**tampilMahasiswaIPKdiAtas(double ipkBatas)**) yang ada di dalam binary search tree.
4. Modifikasi class **BinaryTreeArray00** di atas, dan tambahkan :
 - method **add(Mahasiswa data)** untuk memasukkan data ke dalam binary tree
 - method **traversePreOrder()**

Kode Program

• Class Mahasiswa01

```
package Pertemuan14;
```

```
public class Mahasiswa01 {
```

```
    String nim;  
    String nama;  
    String kelas;  
    double ipk;
```

```
    Mahasiswa01() {  
  
    }  
}
```

```
    Mahasiswa01(String nim, String nama, String kelas, double ipk) {  
        this.nim = nim;  
        this.nama = nama;  
        this.kelas = kelas;  
        this.ipk = ipk;  
    }  
}
```

```
    void tampilInformasi() {  
        System.out.println("NIM: " + this.nim + " " + "Nama: " + this.nama + " " + "Kelas: " +  
            this.kelas + " "  
            + "IPK: " + this.ipk);  
    }  
}
```

- **Class Node01**

```
package Pertemuan14;
```

```
public class Node01 {  
  
    Mahasiswa01 mahasiswa;  
    Node01 left, right;  
  
    Node01() {  
  
    }  
  
    Node01(Mahasiswa01 mahasiswa) {  
        this.mahasiswa = mahasiswa;  
        left = right = null;  
    }  
}
```

- **Class BinaryTree01**

```
package Pertemuan14;
```

```
public class BinaryTree01 {  
  
    Node01 root;  
  
    BinaryTree01() {  
        root = null;  
    }  
  
    boolean isEmpty() {  
        return root == null;  
    }  
  
    void add(Mahasiswa01 mahasiswa) {  
        Node01 newNode = new Node01(mahasiswa);  
        if (isEmpty()) {  
            root = newNode;  
        } else {  
            Node01 current = root;  
            Node01 parent = null;  
            while (true) {  
                parent = current;  
                if (mahasiswa.ipk < current.mahasiswa.ipk) {  
                    current = current.left;  
                    if (current == null) {  
                        parent.left = newNode;  
                        return;  
                    }  
                } else {  
                    current = current.right;  
                    if (current == null) {  
                        parent.right = newNode;  
                        return;  
                    }  
                }  
            }  
        }  
    }  
}
```

```

    }
  }
}

```

```

boolean find(double ipk) {
    boolean result = false;
    Node01 current = root;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            result = true;
            break;
        } else if (ipk > current.mahasiswa.ipk) {
            current = current.right;
        } else {
            current = current.left;
        }
    }
    return result;
}

```

```

void traversePreOrder(Node01 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

```

```

void traverseInOrder(Node01 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

```

```

void traversePostOrder(Node01 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

```

```

Node01 getSuccessor(Node01 del) {
    Node01 successor = del.right;
    Node01 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
}

```

```

    }
    return successor;
}

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    // cari node (current) yang akan dihapus
    Node01 parent = root;
    Node01 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }

    // penghapusan
    if (current == null) {
        System.out.println("Data tidak ditemukan");
        return;
    } else {
        // jika tidak ada anak (leaf), maka node dihapus
        if (current.left == null && current.right == null) {
            if (current == root) {
                root = null;
            } else {
                if (isLeftChild) {
                    parent.left = null;
                } else {
                    parent.right = null;
                }
            }
        }
        } else if (current.left == null) { // jika hanya punya 1 anak (kanan)
            if (current == root) {
                root = current.right;
            } else {
                if (isLeftChild) {
                    parent.left = current.right;
                } else {
                    parent.right = current.right;
                }
            }
        } else if (current.right == null) { // jika hanya punya 1 anak (kiri)
            if (current == root) {

```

```

        root = current.left;
    } else {
        if (isLeftChild) {
            parent.left = current.left;
        } else {
            parent.right = current.left;
        }
    }
} else { // jika punya 2 anak
    Node01 successor = getSuccessor(current);
    /*
    * System.out.println("Jika 2 anak, current = ");
    * successor.mahasiswa.tampilInformasi();
    */
    if (current == root) {
        root = successor;
    } else {
        if (isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}
}

```

```

Node01 addRekursif(Node01 node, Mahasiswa01 mahasiswa) {
    Node01 newNode = new Node01(mahasiswa);
    if (node == null) {
        return newNode;
    }
    if (mahasiswa.ipk < node.mahasiswa.ipk) {
        node.left = addRekursif(node.left, mahasiswa);
    } else {
        node.right = addRekursif(node.right, mahasiswa);
    }
    return node;
}

```

```

void cariMinIpk(Node01 nodeMin) {
    Node01 min = root;
    while (nodeMin.left != null) {
        nodeMin = nodeMin.left;
        min = nodeMin;
    }
    min.mahasiswa.tampilInformasi();
}

```

```

void cariMaxIpk(Node01 nodeMin) {
    Node01 max = root;
    while (nodeMin.right != null) {
        nodeMin = nodeMin.right;
        max = nodeMin;
    }
}

```

```

    }
    max.mahasiswa.tampilInformasi();
}

void tampilMahasiswaIPKdiAtas(Node01 node, double ipkBatas) {
    if (node == null) {
        return;
    }
    if (node.mahasiswa.ipk > ipkBatas) {
        tampilMahasiswaIPKdiAtas(node.left, ipkBatas);
        node.mahasiswa.tampilInformasi();
        tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
    } else {
        tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
    }
}
}
}

```

• Class BinaryTreeMain01

```
package Pertemuan14;
```

```

public class BinaryTreeMain01 {

    public static void main(String[] args) {
        BinaryTree01 bst = new BinaryTree01();

        bst.add(new Mahasiswa01("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa01("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa01("244160185", "Candra", "C", 3.21));
        bst.add(new Mahasiswa01("244160220", "Dewi", "B", 3.54));

        System.out.println("\nDaftar semua mahasiswa (in order traversal) :");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("Cari mahasiswa dengan ipk: 3.54 : ");
        String hasilCari = bst.find(3.54) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        System.out.print("Cari mahasiswa dengan ipk: 3.22 : ");
        hasilCari = bst.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        bst.add(new Mahasiswa01("244160131", "Devi", "A", 3.72));
        bst.add(new Mahasiswa01("244160205", "Ehsan", "D", 3.37));
        bst.add(new Mahasiswa01("244160170", "Fizi", "B", 3.46));
        System.out.println("\nDaftar semua mahasiwa setelah penambahan 3 mahasiswa:");
        System.out.println("InOrder Traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println("PreOrder Traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println("PostOrder Traversal:");
        bst.traversePostOrder(bst.root);
    }
}

```

```

        System.out.println("\nPenghapusan data mahasiswa");
        bst.delete(3.57);
        System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPenambahan data mahasiswa");
        bst.addRekursif(bst.root, new Mahasiswa01("244160170", "Fafa", "C", 3.38));
        System.out.println("NIM: 244160170 Nama: Fafa Kelas: C IPK: 3.38");

        System.out.println("\nDaftar semua mahasiswa setelah penambahan 1 mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nData mahasiswa dengan ipk paling kecil");
        bst.cariMinIpk(bst.root);
        System.out.println("Data mahasiswa dengan ipk paling besar");
        bst.cariMaxIpk(bst.root);

        System.out.println("\nDaftar mahasiswa dengan IPK diatas 3.50");
        bst.tampilMahasiswaIPKdiAtas(bst.root, 3.50);
    }
}

```

• Class BinaryTreeArray01

```

package Pertemuan14;

public class BinaryTreeArray01 {
    Mahasiswa01[] dataMahasiswa;
    int idxLast;

    BinaryTreeArray01() {
        this.dataMahasiswa = new Mahasiswa01[10];
    }

    void populateData(Mahasiswa01 dataMhs[], int idxLast) {
        this.dataMahasiswa = dataMhs;
        this.idxLast = idxLast;
    }

    void traverseInOrder(int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                traverseInOrder(2 * idxStart + 1);
                dataMahasiswa[idxStart].tampilInformasi();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }

    void add(Mahasiswa01 data) {
        if (idxLast < dataMahasiswa.length - 1) {
            idxLast++;
            dataMahasiswa[idxLast] = data;
        }
    }
}

```

```

        System.out.println("NIM: " + data.nim + " Nama: " + data.nama + " Kelas: " +
data.kelas + " IPK:" + data.ipk);
    } else {
        System.out.println("Data mahasiswa sudah penuh.");
    }
}

void traversePreOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            dataMahasiswa[idxStart].tampilInformasi();
            traversePreOrder(2 * idxStart + 1);
            traversePreOrder(2 * idxStart + 2);
        }
    }
}
}
}
}

```

• Class BinaryTreeArrayMain01

```
package Pertemuan14;
```

```

public class BinaryTreeArrayMain01 {
    public static void main(String[] args) {
        BinaryTreeArray01 bta = new BinaryTreeArray01();
        Mahasiswa01 mhs1 = new Mahasiswa01("244160121", "Ali", "A", 3.57);
        Mahasiswa01 mhs2 = new Mahasiswa01("244160185", "Candra", "C", 3.41);
        Mahasiswa01 mhs3 = new Mahasiswa01("244160221", "Badar", "B", 3.75);
        Mahasiswa01 mhs4 = new Mahasiswa01("244160220", "Dewi", "B", 3.35);

        Mahasiswa01 mhs5 = new Mahasiswa01("244160131", "Devi", "A", 3.48);
        Mahasiswa01 mhs6 = new Mahasiswa01("244160205", "Ehsan", "D", 3.61);
        Mahasiswa01 mhs7 = new Mahasiswa01("244160170", "Fizi", "B", 3.86);

        Mahasiswa01[] dataMahasiswa = { mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7, null,
            null, null };
        int idxLast = 6;
        bta.populateData(dataMahasiswa, idxLast);
        System.out.println("\nInOrder Traversal Mahasiswa: ");
        bta.traverseInOrder(0);

        System.out.println("\nMenambah data mahasiswa.");
        bta.add(new Mahasiswa01("244160111", "Fafa", "C", 3.22));

        System.out.println("\nPreOrder Traversal Mahasiswa: ");
        bta.traversePreOrder(0);
    }
}
}

```

Hasil Run Terminal

Penambahan data mahasiswa

NIM: 244160170 Nama: Fafa Kelas: C IPK: 3.38

Daftar semua mahasiswa setelah penambahan 1 mahasiswa (in order traversal):

NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21

NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37

NIM: 244160170 Nama: Fafa Kelas: C IPK: 3.38

NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46

NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54

NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Data mahasiswa dengan ipk paling kecil

NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21

Data mahasiswa dengan ipk paling besar

NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Daftar mahasiswa dengan IPK diatas 3.50

NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54

NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

PS C:\PENYIMPANAN\Documents\G\PraktikumAlgoritmaDanStrukturData>

BinaryTreeArray

Menambah data mahasiswa.

NIM: 244160111 Nama: Fafa Kelas: C IPK:3.22

PreOrder Traversal Mahasiswa:

NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41

NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35

NIM: 244160111 Nama: Fafa Kelas: C IPK: 3.22

NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48

NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75

NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61

NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86

PS C:\PENYIMPANAN\Documents\G\PraktikumAlgoritmaDanStrukturData>